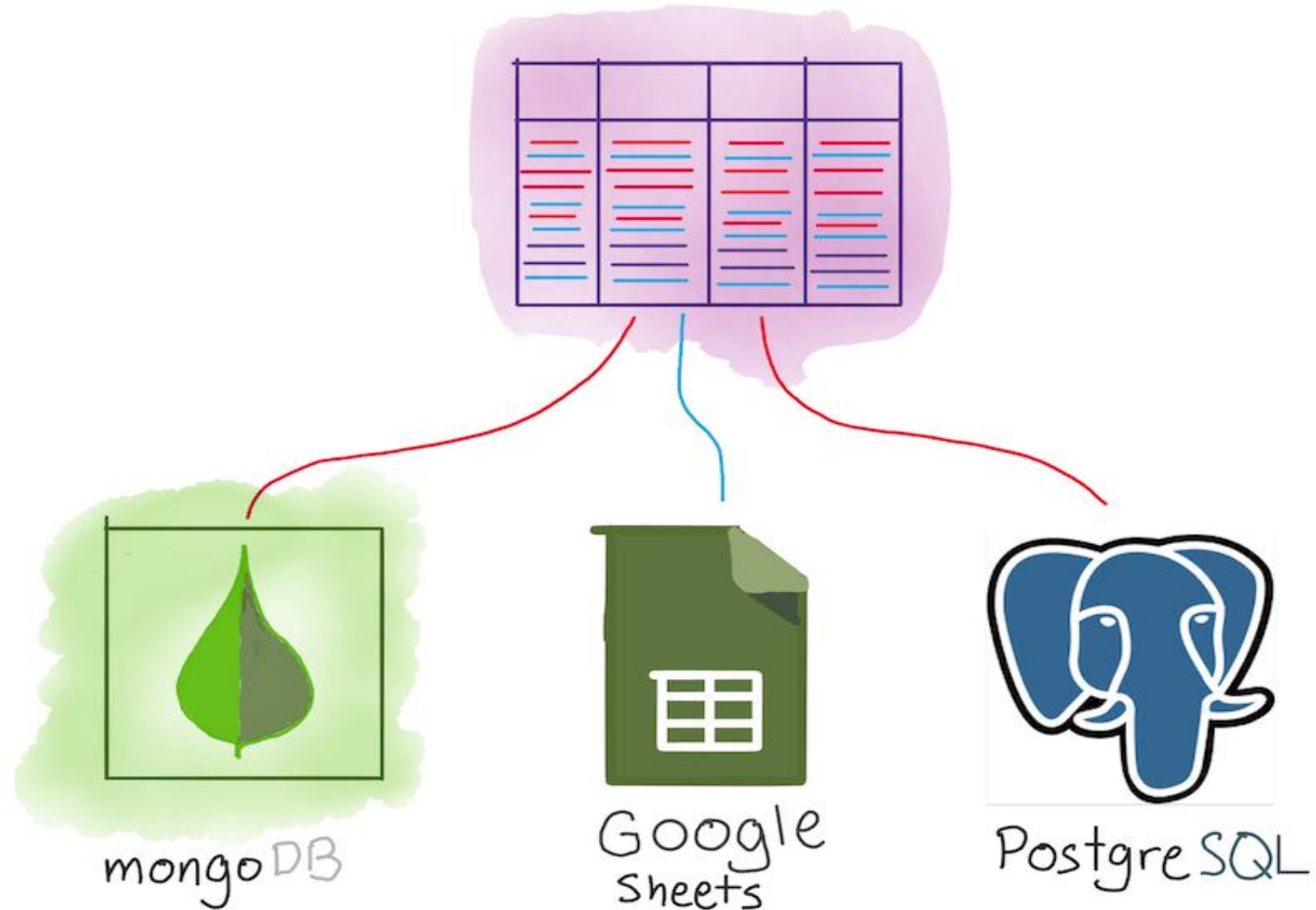


Source Data Model Design

DATA MODEL



Na vandaag...

Weet/ken je...

Kan je...

Wat Source Data Modellen en Datawarehouses zijn en waarvoor ze gebruikt worden

Welke 2 soorten databases er zijn en wanneer je welke soort het beste kan inzetten.

Uit welke stappen de “datapijplijn” bestaat tussen operationele databases enerzijds en datawarehouses anderzijds

Wat database-overschrijdende associaties zijn

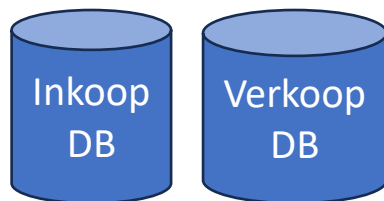
Een Source Data Model ontwerpen uit verschillende operationele databronnen

Feiten & dimensies herkennen in reeds gemaakte Source Data Modellen

Uit Source Data Modellen conclusies trekken over analysemogelijkheden

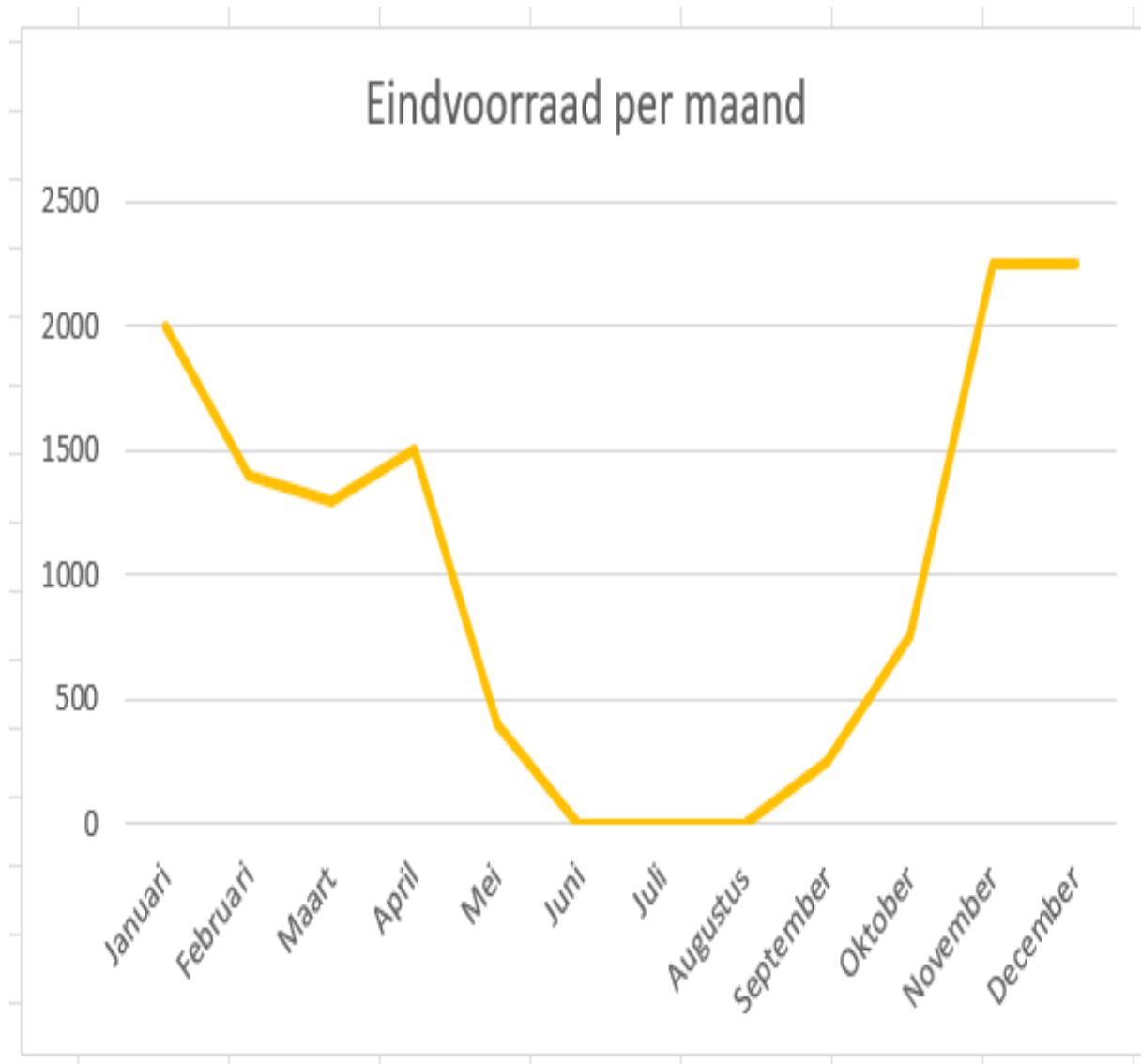
HC + WC





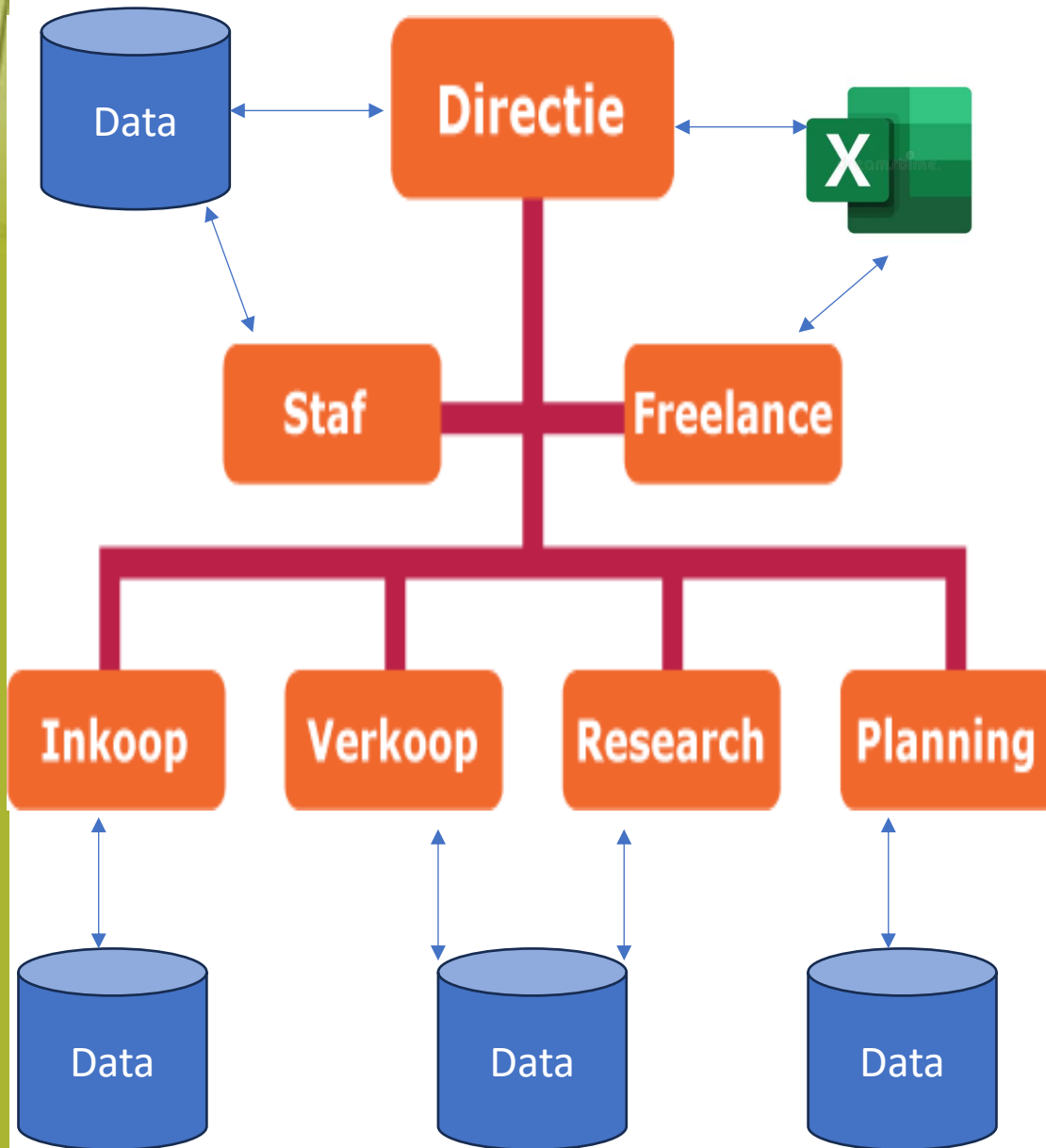
Welke beslissing?

Maand	Voorraad begin	Ingekocht	Verkocht	Voorraad eind
Januari	1000	1500	500	2000
Februari	2000	400	1000	1400
Maart	1400	700	800	1300
April	1300	500	300	1500
Mei	1500	200	1300	400
Juni	400	1000	1400	0
Juli	0	2000	2500	0
Augustus	-500	2000	1750	0
September	-250	2500	2000	250
Oktober	250	2000	1500	750
November	750	2000	500	2250
December	2250	500	500	2250



Welk advies zou je geven aan deze willekeurige winkel?

Databronnen & -versnippering



Data-analyses willen prestaties meten van het **volledige** bedrijf, zodat mensen aan de hand van statistiek de juiste beslissingen kunnen nemen.

Echter: in veel organisaties ligt data versnipperd over meerdere databases, Excelbestanden en andere soorten databronnen.

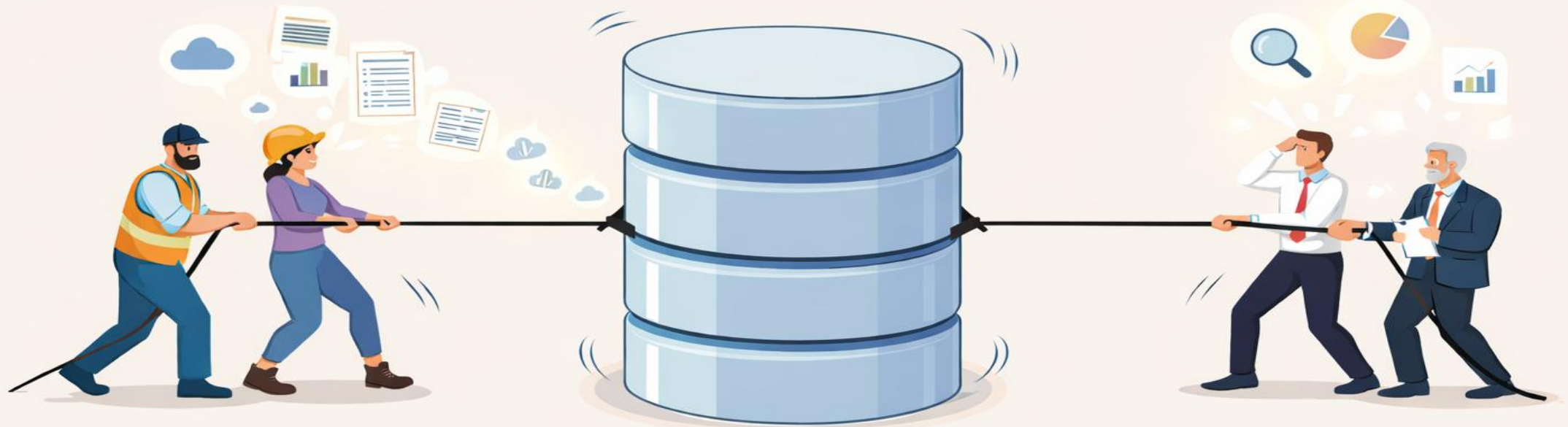
Data Engineering / Business Intelligence:
“Hoe kunnen we onze data het beste centraliseren, zodat er één toegangspunt voor alle analyses ontstaat?”

Vroeger: tweeledig gebruik van databronnen

Operationele medewerkers
willen data toevoegen

Databronnen moesten aan 2 tegenstrijdige
behoeften tegelijkertijd voldoen!
Toen de hoeveelheid data explosief steeg
("Big Data") ging hun performance sterk achteruit

Managers & directeurs
willen data analyseren



- Veel kleine transacties
- Inserts, updates & deletes
- Lage latency nodig: data moet snel opgeslagen kunnen worden.

- **Traagheid**
- **Queries die elkaar "locken"**
- **De mate van normalisatie is niet optimaal voor één van beide partijen (zie volgende slides)**

- Grote selects over miljoenen rijen.
- Complexe joins.
- Complexe aggregaties.

Voor- en nadelen van databasennormalisatie

Hoe genormaliseerder een datastructuur is, hoe meer verschillende tabellen er zijn waarin data is “opgeknipt”

Medicatie

Naam	Gebruiken bij	Wijze van toedienen
Midazolam	Epilepsie	1 spray per neusgat (dus in totaal 2)
Paracetamol	Pijn	Oplossen in beker water
Cetirizine	Hooikoorts	Oplossen in beker water

Leden

Naam	Leeftijd	Medicatie
Anna	16	Midazolam
Bernard	16	Paracetamol
Carlo	18	Midazolam
Diana	18	Cetirizine
Erik	20	Cetirizine
Flora	20	Midazolam



Sterke mate van normalisatie -> Registratie-efficiëntie:
Van elk medicijn hoeft slechts één keer alle data geregistreerd te worden

Voor- en nadelen van databasennormalisatie

Hoe genormaliseerder een datastructuur is, hoe meer verschillende tabellen er zijn waarin data is “opgeknipt”

Medicatie

Naam	Gebruiken bij	Wijze van toedienen
Midazolam	Epilepsie	1 spray per neusgat (dus in totaal 2)
Paracetamol	Pijn	Oplossen in beker water
Cetirizine	Hooikoorts	Oplossen in beker water

Leden

Naam	Leeftijd	Medicatie
Anna	16	Midazolam
Bernard	16	Paracetamol
Carlo	18	Midazolam
Diana	18	Cetirizine
Erik	20	Cetirizine
Flora	20	Midazolam



Carlo krijgt opeens een epileptische aanval

Wat moeten we doen?!

Elke seconde telt bij een epileptische aanval!

Tóch moet er in 2 verschillende tabellen gekeken worden, dit kost tijd en moeite!

Genormaliseerde datastructuren zijn wél registratie-, maar **niet analyse-efficient!**

Welke datastructuur was hier beter geweest?

Een minder genormaliseerde datastructuur, omdat hierin alle data zoveel mogelijk samengevoegd is in één tabel.

Je bereikt tijdswinst, doordat je nú slechts in één tabel hoeft te kijken.

Naam	Leeftijd	Medicatie	Gebruiken bij	Wijze van toedienen
Anna	16	Midalozam	Epilepsie	1 spray per neusgat (dus in totaal 2)
Bernard	16	Paracetamol	Pijn	Oplossen in beker water
Carlo	18	Midalozam	Epilepsie	1 spray per neusgat (dus in totaal 2)
Diana	18	Cetirizine	Hooikoorts	Oplossen in beker water
Erik	20	Cetirizine	Hooikoorts	Oplossen in beker water
Flora	20	Midalozam	Epilepsie	1 spray per neusgat (dus in totaal 2)



Carlo krijgt een epileptische aanval

Wat moeten we doen?!

Minder genormaliseerde datastructuren zijn wél analyse-, maar

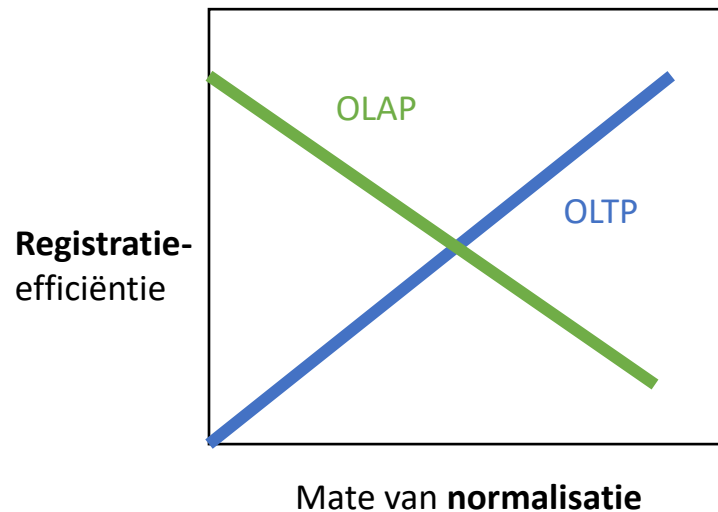
niet registratie-efficiënt, vanwege mogelijke **redundantie**:

Soms moet bepaalde data op meerdere plekken opgeslagen worden

OLTP versus OLAP

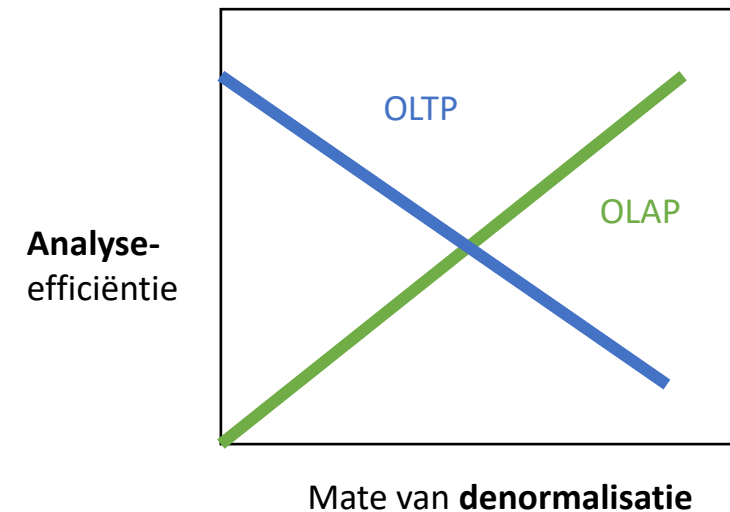
Online Transactional Processing

- Databases waarin data vooral **geregistreerd** en slechts beperkt geanalyseerd wordt.
- Sterk genormaliseerd (vaak 3NV) met weinig redundantie, om te voorkomen dat je bij het registreren van één entiteit ook nog andere entiteiten moet aanmaken.



Online Analytical Processing

- Databases waarin data vooral **geanalyseerd** en slechts beperkt geregistreerd wordt.
- Minder sterk genormaliseerd met veel redundantie, om te voorkomen dat je bij het zoeken naar data teveel tabellen moet raadplegen.



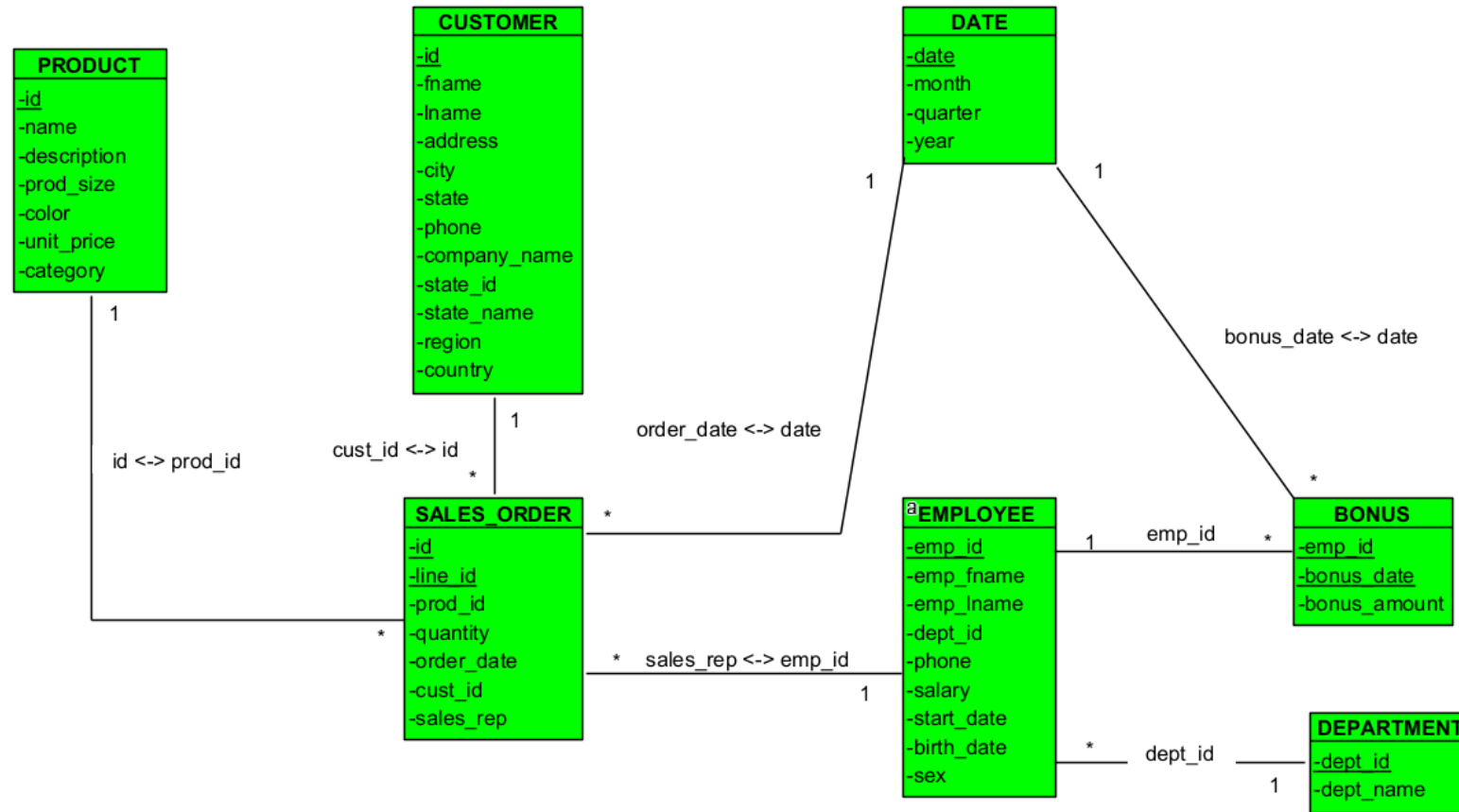
Voorbeeld OLTP met registratie-efficiëntie



Efficiënties:

- Registratie-efficiëntie: is **hoog**. Bij het aanmaken van een nieuwe “Customer”, zijn de data van zijn/haar “State” vooraf al ingevuld.
- Analyse-efficiëntie: is **laag**. Om elke “Customer” aan de juiste “State” te relateren, moeten twee tabellen geraadpleegd worden.

Voorbeeld OLAP met analyse-efficiëntie

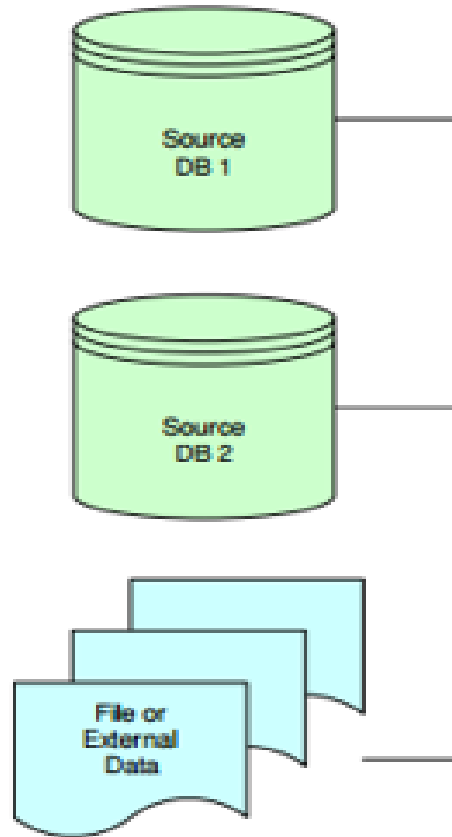


Efficiënties:

- Registratie-efficiëntie: is **laag**. Bij het aanmaken van een nieuwe “Customer” moet óók alle “State”-data opnieuw ingevuld worden. Zodoende kan elke staat meerdere keren voorkomen in deze tabel: redundantie.
- Analyse-efficiëntie: is **hoog**. Om elke “Customer” aan de juiste “State” te relateren, hoeft er slechts één tabel geraadpleegd te worden.

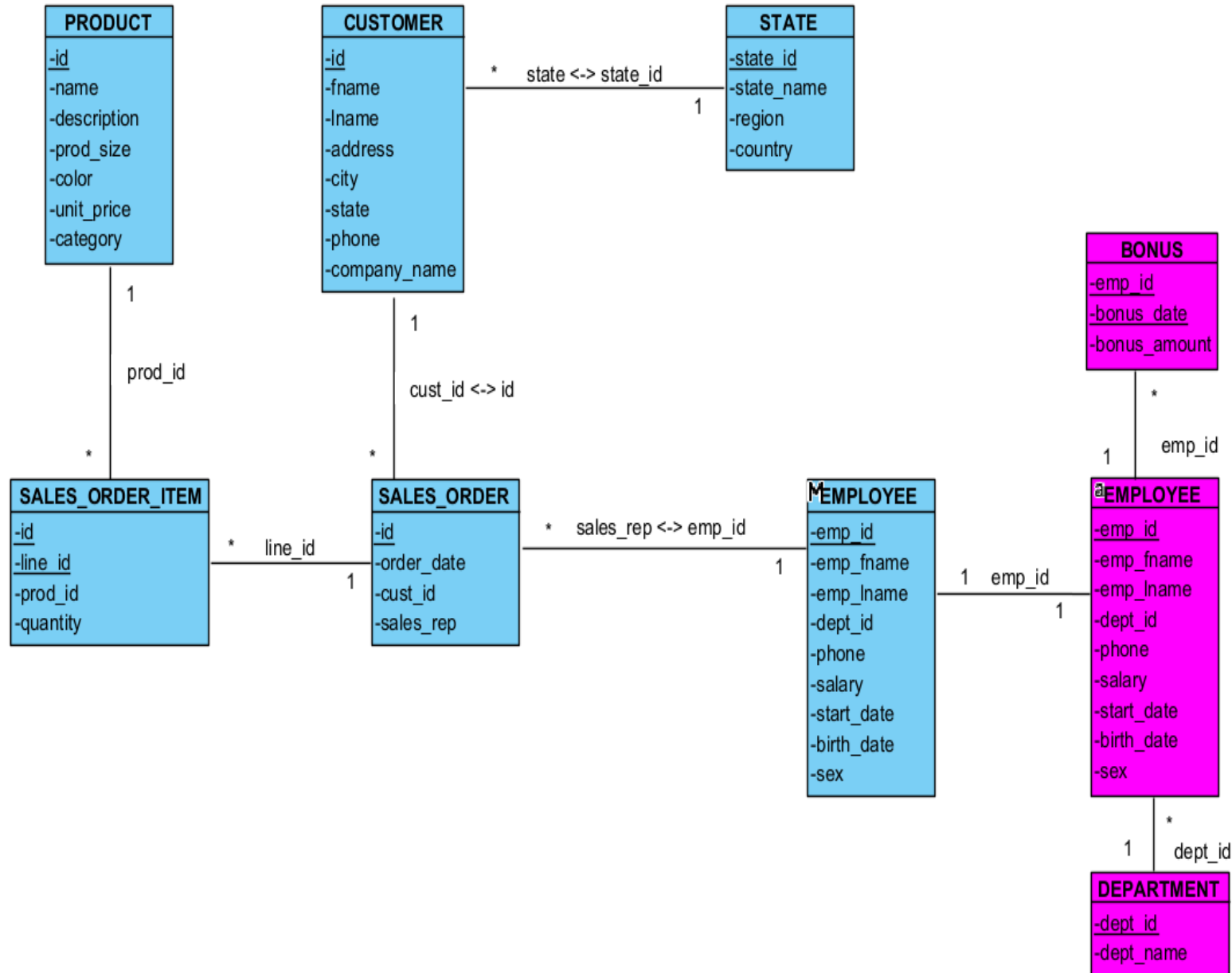
Nu: een volledig geïmplementeerde “datapijplijn”

Databronnen (OLTP)



Vroeger moest er een keuze gemaakt worden: OLTP- óf OLAP-structuur
Tegenwoordig kunnen beide structuren effectief gecombineerd worden in één overkoepelende,
en vaak geautomatiseerde, “datapijplijn”

Het Source Data Model (OLTP)

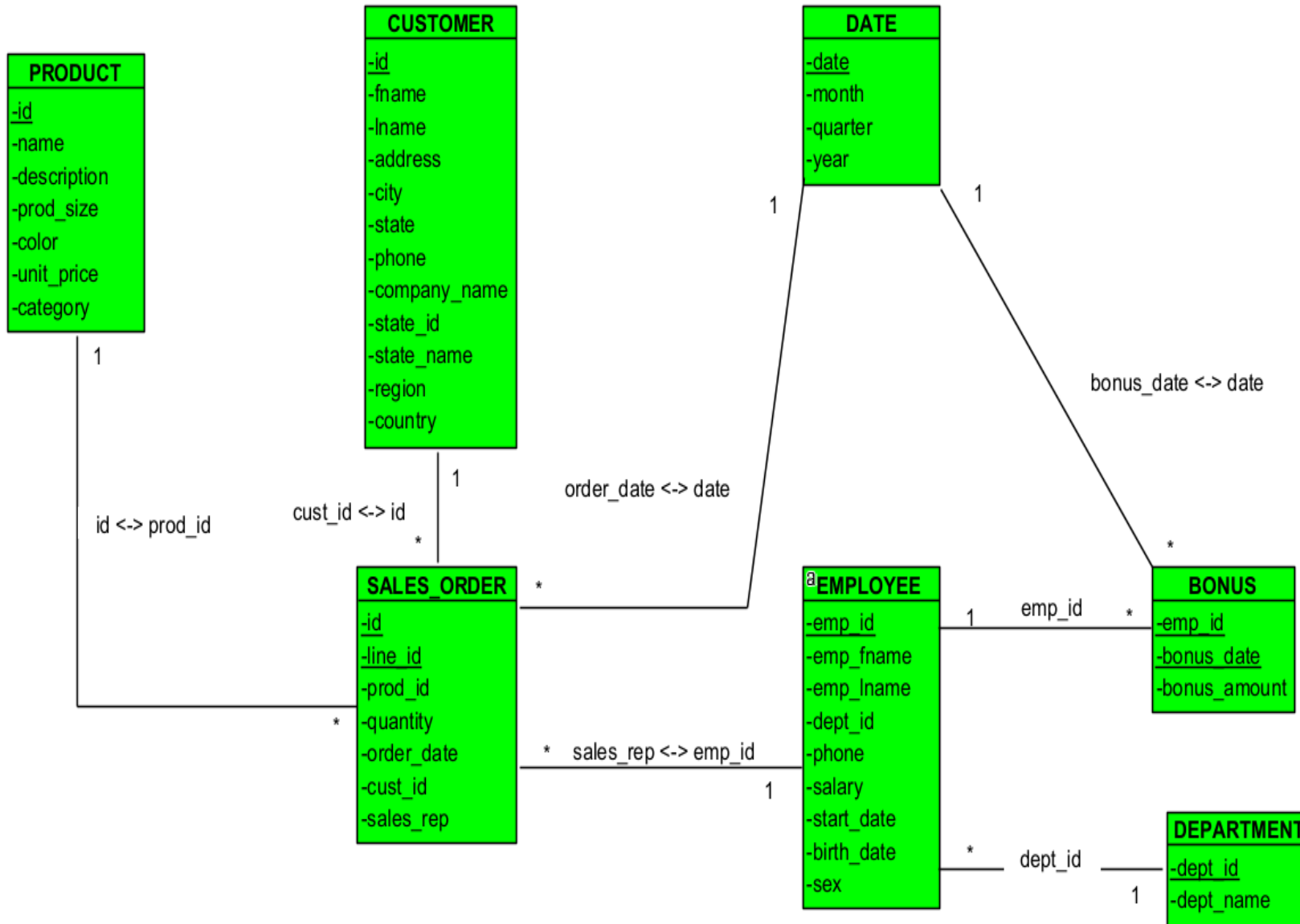


Een Source Data Model is een database waarin data uit meerdere databronnen met elkaar wordt gecombineerd volgens hun originele datastructuur.

In dit voorbeeld is de inhoud van twee databases (**sales_order** en **hrm**) gecombineerd in één nieuwe database.

Deze datastructuur is nu nog in OLTP-vorm: sterk genormaliseerd. Deze wordt hierna geoptimaliseerd voor analyse: de OLAP-vorm.

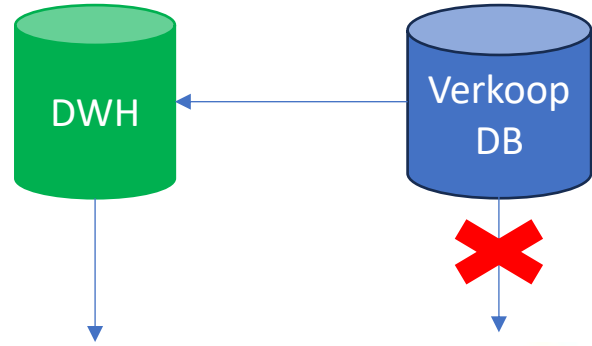
Het Data Warehouse (OLAP)



Inmiddels is de structuur geschikt gemaakt voor analyse d.m.v. dimensioneel modelleren

Dit is het eindproduct van datapijplijnen: een datawarehouse op basis waarvan datadashboards gevuld kunnen worden

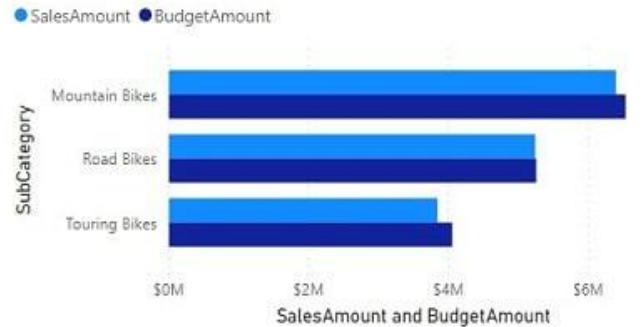
Voorwaartse
Data Lineage



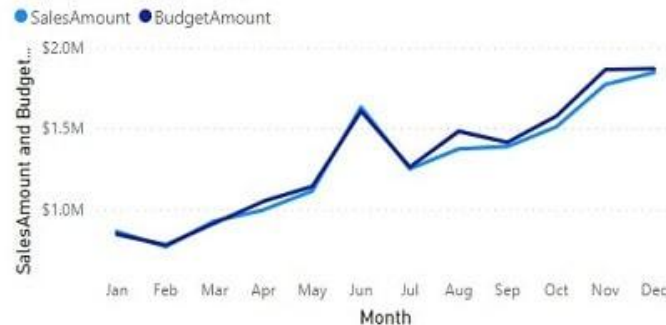
Bikeshop Sales



SalesAmount and BudgetAmount by Category and SubCategory



SalesAmount and BudgetAmount by Month



Voorbeeld dashboard

Achterwaartse
Data Lineage

\$15M

SalesAmountMeas

\$16M

BudgetAmountMeas

-2.3%

DiffPercent

SalesAmountMeas by Occupation



ProductName SalesAmountMeas

Mountain-200 Black, 42	\$979,961
Mountain-200 Silver, 38	\$979,036
Mountain-200 Black, 46	\$961,601
Mountain-200 Black, 38	\$954,716
Mountain-200 Silver, 46	\$930,316
Mountain-200 Silver, 42	\$909,436
Touring-1000 Blue, 46	\$421,980
Road-350-W Yellow, 40	\$418,444
Touring-1000 Yellow, 46	\$410,060
Road-350-W Yellow, 42	\$399,733

SalesAmount by Country



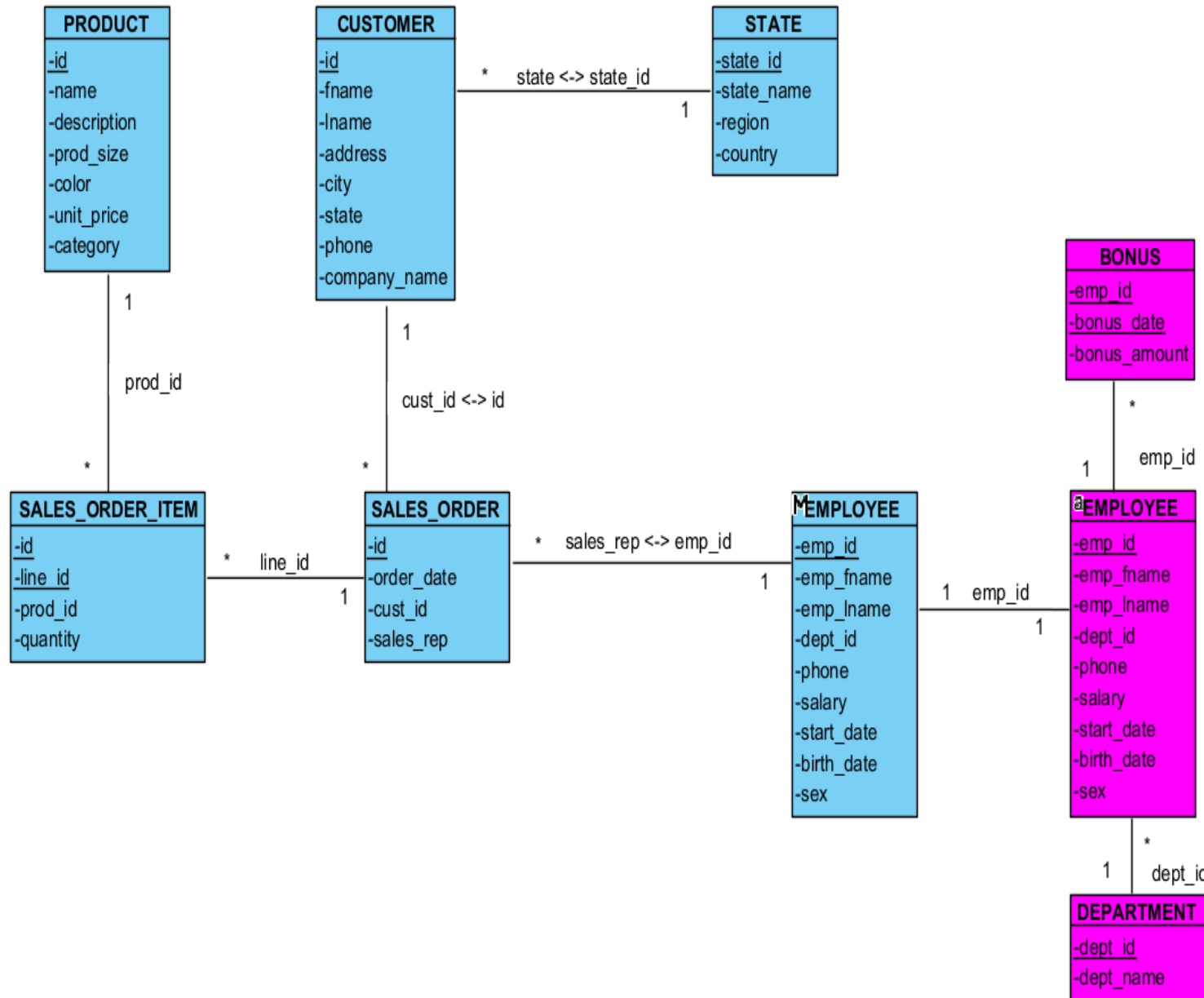
Op basis van dashboards worden de juiste statistische inzichten verkregen, waarmee de beste beslissingen kunnen worden genomen

Alléén het data warehouse heeft een directe verbinding met dashboards:
“single point of truth”

Data Lineages:

- Voorwaarts: waarvoor gebruiken we deze data?
- Achterwaarts: waar komt deze data vandaan?

SDM's en database-overschrijdende associaties



Losstaande databases kan je combineren tot SDM's door te zoeken naar database-overschrijdende associaties: koppelingen tussen tabellen die...

- Wel eenzelfde entiteit beschrijven.
- Niet tot dezelfde database behoren.

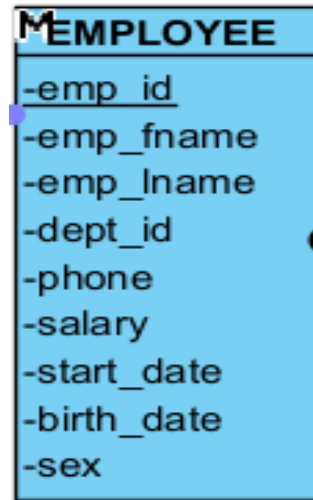
In dit geval zien we dat er 1 entiteit is die in beide databases voorkomt: "EMPLOYEE". Tussen hen komt er dus een nieuwe associatie.

Dit zijn bijna altijd 1-op-1-associaties, "meer" komt hierbij bijna nooit voor.

Wanneer is de multipliciteit 1 en wanneer 0..1?

1-1-database-overschrijdende associaties

Database: Sales_Order

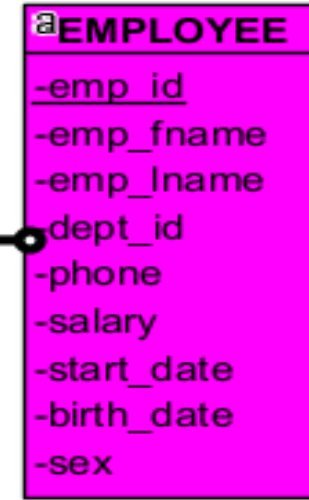


1

emp id

1

Database: HRM



Emp_id	Emp_fname	Emp_lname
1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven

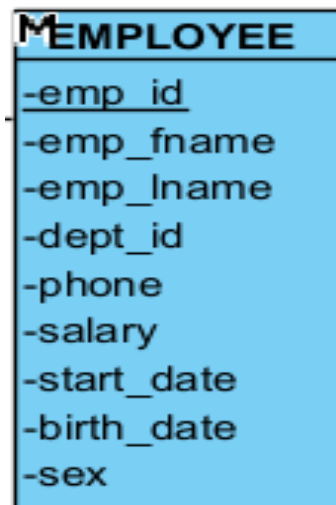
Emp_id	Emp_fname	Emp_lname
1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven

Constateringen:

- Elke **employee** kan gerelateerd worden aan een **employee**.
- Elke **employee** kan gerelateerd worden aan een **employee**.
- Eindconclusie: de inhoud van beide tabellen is exact hetzelfde! De multiplicititeit wordt aan beide kanten 1!

1-0..1-database-overschrijdende associaties

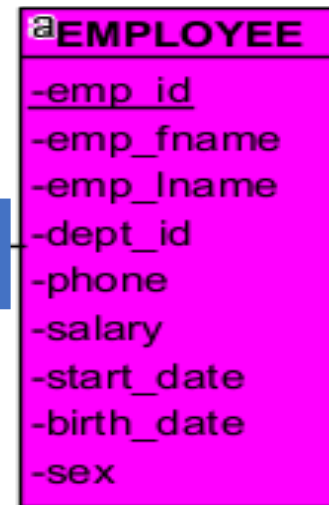
Database: Sales_Order



1

emp_id

Database: HRM



0..1

Emp_id	Emp_fname	Emp_lname
1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven

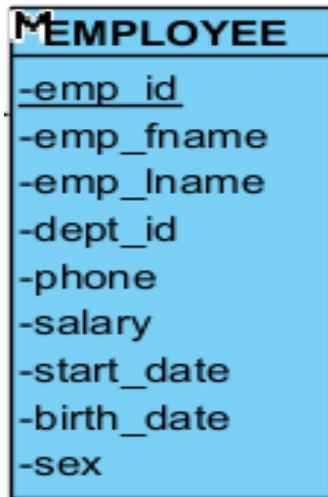
Emp_id	Emp_fname	Emp_lname
1	Henk	de Vries
2	Marleen	Langelaan

Constateringen:

- Elke **employee** kan niet altijd gerelateerd worden aan een **employee**.
- Elke **employee** kan wel altijd gerelateerd worden aan een **employee**.
- Eindconclusie: aan de roze kant moet 0..1 komen te staan, aan de andere kant 1!

0..1-0..1-database-overschrijdende associaties

Database: Sales_Order

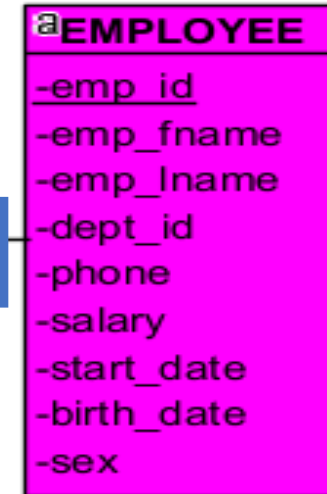


0..1

emp_id

0..1

Database: HRM



Emp_id	Emp_fname	Emp_lname
1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven

Emp_id	Emp_fname	Emp_lname
1	Henk	de Vries
2	Marleen	Langelaan
3	Frida	de Haan

Constateringen:

- Elke **employee** kan niet altijd gerelateerd worden aan een **employee**.
- Elke **employee** kan niet altijd gerelateerd worden aan een **employee**.
- Eindconclusie: aan beide kanten moet 0..1 komen te staan!

Dimensies, feiten en meetwaarden onderscheiden

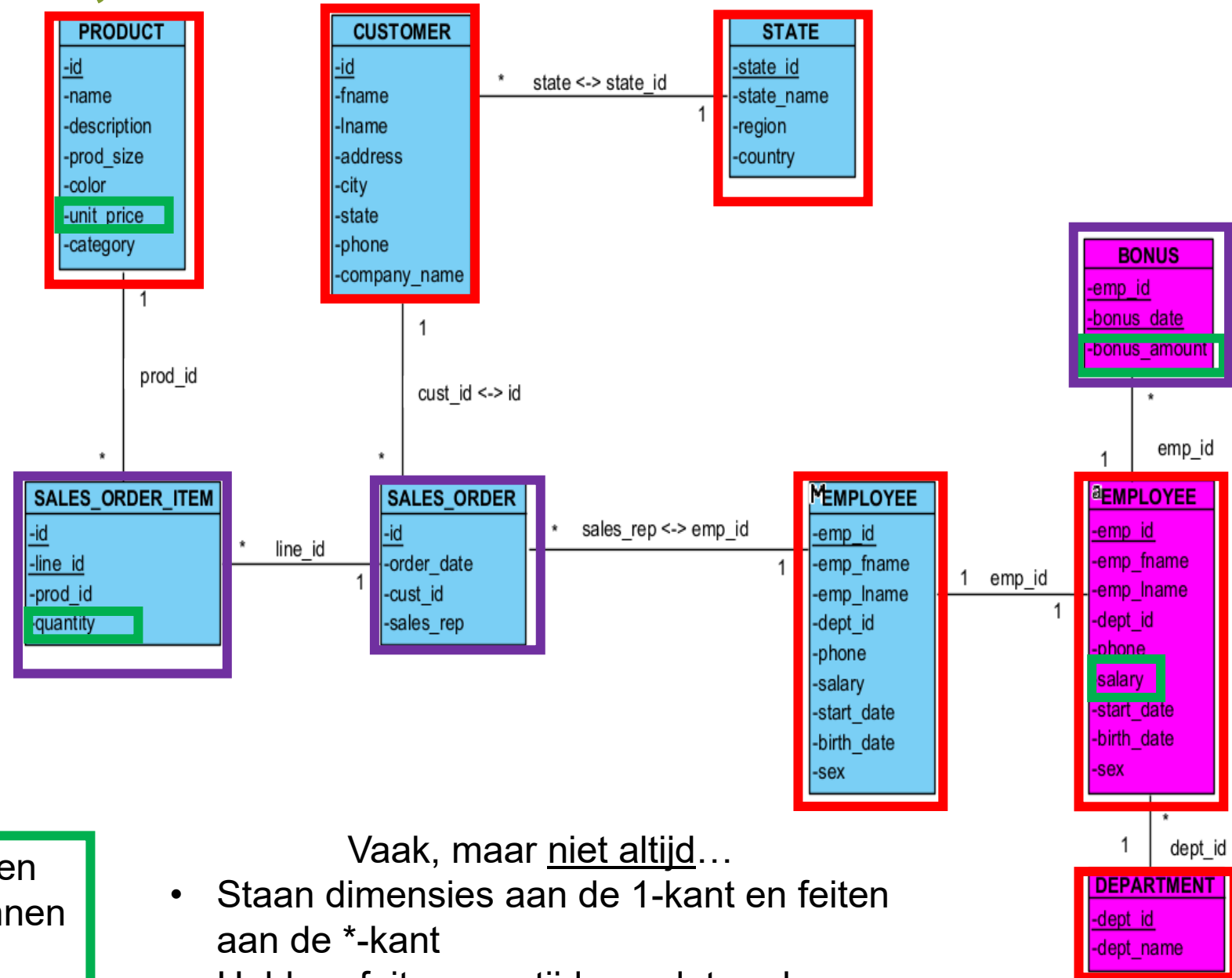
Een dimensie is...

- Een mens
- Een ding

Een feit is...

- Een proces
- Een activiteit
- Een gebeurtenis

Een meetwaarde is een numeriek attribuut binnen een feit of dimensie



Vaak, maar niet altijd...

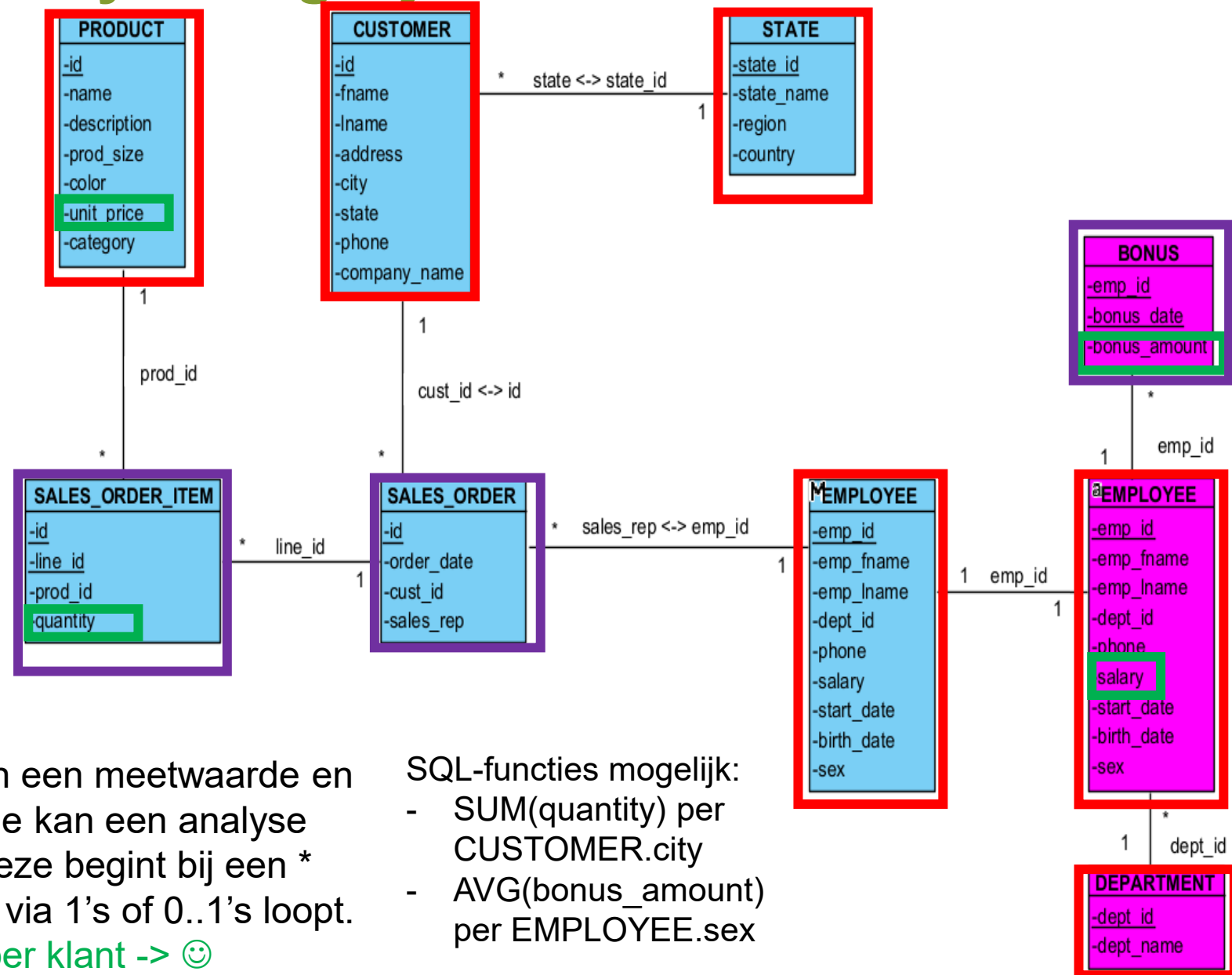
- Staan dimensies aan de 1-kant en feiten aan de *-kant
- Hebben feiten een tijdsgelateerd attribuut (datums, timestamps, enzovoort)

Analysemogelijkheden inventariseren

Dimensie

Feit

Meetwaarde



Elke combinatie tussen een meetwaarde en een niet-meetwaarde kan een analyse opleveren, zolang deze begint bij een * (meer) en voor de rest via 1's of 0..1's loopt.

- Verkoopen per klant -> 😊
- Bonussen per klant -> ☹️

SQL-functies mogelijk:

- SUM(quantity) per CUSTOMER.city
- AVG(bonus_amount) per EMPLOYEE.sex

Enzovoort

Na vandaag...

Weet/ken je...

Kan je...

Wat Source Data Modellen en Datawarehouses zijn en waarvoor ze gebruikt worden

Welke 2 soorten databases er zijn en wanneer je welke soort het beste kan inzetten.

Uit welke stappen de “datapijplijn” bestaat tussen operationele databases enerzijds en datawarehouses anderzijds

Wat database-overschrijdende associaties zijn

Een Source Data Model ontwerpen uit verschillende operationele databronnen

Feiten & dimensies herkennen in reeds gemaakte Source Data Modellen

Uit Source Data Modellen conclusies trekken over analysemogelijkheden

HC + WC

